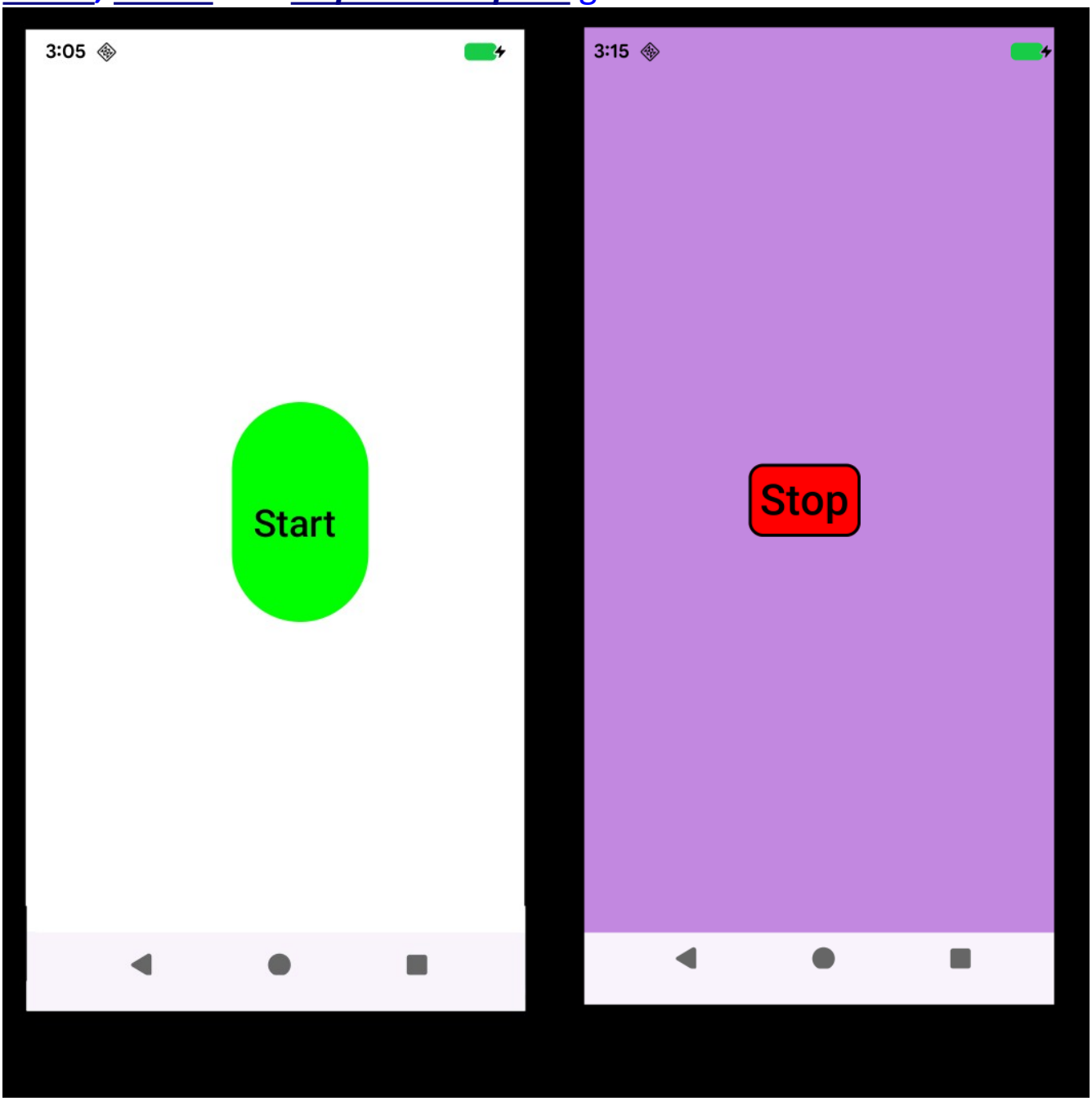
Table of Contents

MVVM Sample App

This sample application shows the proper way to create an ***android smart phone app*** that will not suffer from being too long on the ui thread, or having data scattered everywhere, let's follow the best ***kotlin***, ***mvvm*** and ***Jetpack Compose*** guidelines.



If you press the **start** **button**, a background task,

not on ui thread, changes the background color, every few seconds.

Basic Philosophy

It is important to understand that the **UI thread** must never be stopped or blocked, or your app will display the deadly "*app not responding*" dialog.

MVVM and **Jetpack Compose** also aim to make programming easier by providing a way to have data available from a central location, **ViewModel.kt**, and to help with keeping long running operations off the UI thread, **Back.kt**

There are **three main events** that I would like to cover. What happens at **program startup**, what happens when the **start button** is pressed, and what happens when the **stop** button is pressed.

Hint: Both call toggleRunning.

Quick Start

This guide explains how the android smart phone app,

MVVM, was designed, how it is expected to be used, by android smart phone app developers, such as yourself and how it works under the covers.

This app and guide, were not created to teach you kotlin, jetpack compose or mvvm. Those, and other necessary concepts, like how to install android and how to use github, are briefly covered, but there are many other guides to show and explain these things in greater detail.

This app **was designed** to help anyone use and understand how jetpack compose and mvvm apps are supposed to be designed or created.

It is expected that, after **renaming the sample, MVVM,** you will have the start of a well designed app that is also version controlled using **github**.

Project MVVM has functionality and structure to help with common issues that all projects need, such as:

- communication between the user interface and the data being displayed, in such a way as to not do long running operations on the ui thread, which would lock up the app.
- Logging and other custom operations, via extension functions, **Extensions.kt**
- coroutine management
- Prefs.kt class to handle persisting data easily, when a database is not needed.
- Helpful, custom composeables **ui\composeables\core**
- repo repository to handle long running operations, such as fetching data from the web, other apps or just long running operations.

Clone Sample Project

It is preferable to use android studio to download project Mvvm from github, but you may use **git** directly to download the project files and the apk from **url: https://github.com/pstorli/Mvvm.git**

The apk file **Mvvm.apk***, in the root of the project directory, has already been compiled for you, in case you have not yet installed Android Studio,*

and can be ***side loaded*** *to your android phone.*

The file **Guide.pdf***, also in the project root dir, is* ***this file****.*

a) **Open Android Studio**.

If you don't have android studio, download and install it from:
https://developer.android.com/studio
This sample project was built using:
Otter 2 Feature Drop

b) **File → Close Project**

Do above action for all open projects.

c) Press the **"Clone Repository"** button.

d) Enter url and destination directory.

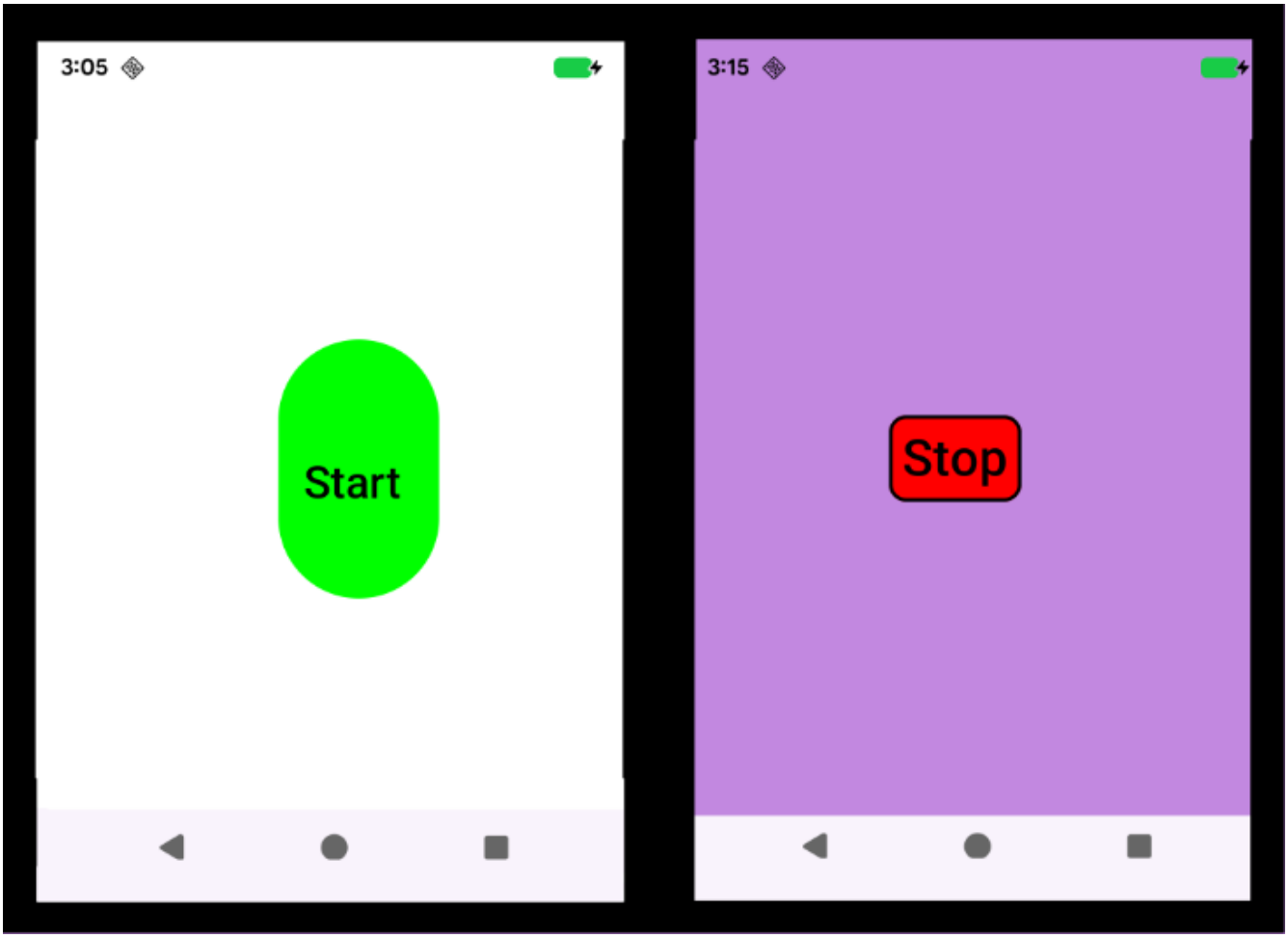
Note: **Parent Directory** does not have to be: *C:\github*
and **Name** does not have to be *MySample,*
use any name for your new project, just no spaces.

Enter the following, ***case sensitive!***
URL: https://github.com/pstorli/MVVM.git
Directory: C:\github\MySample

e) **To run application, Run → Run 'app'**
or **Run → Debug 'menu'**

Connect phone with USB cable or use emulator.

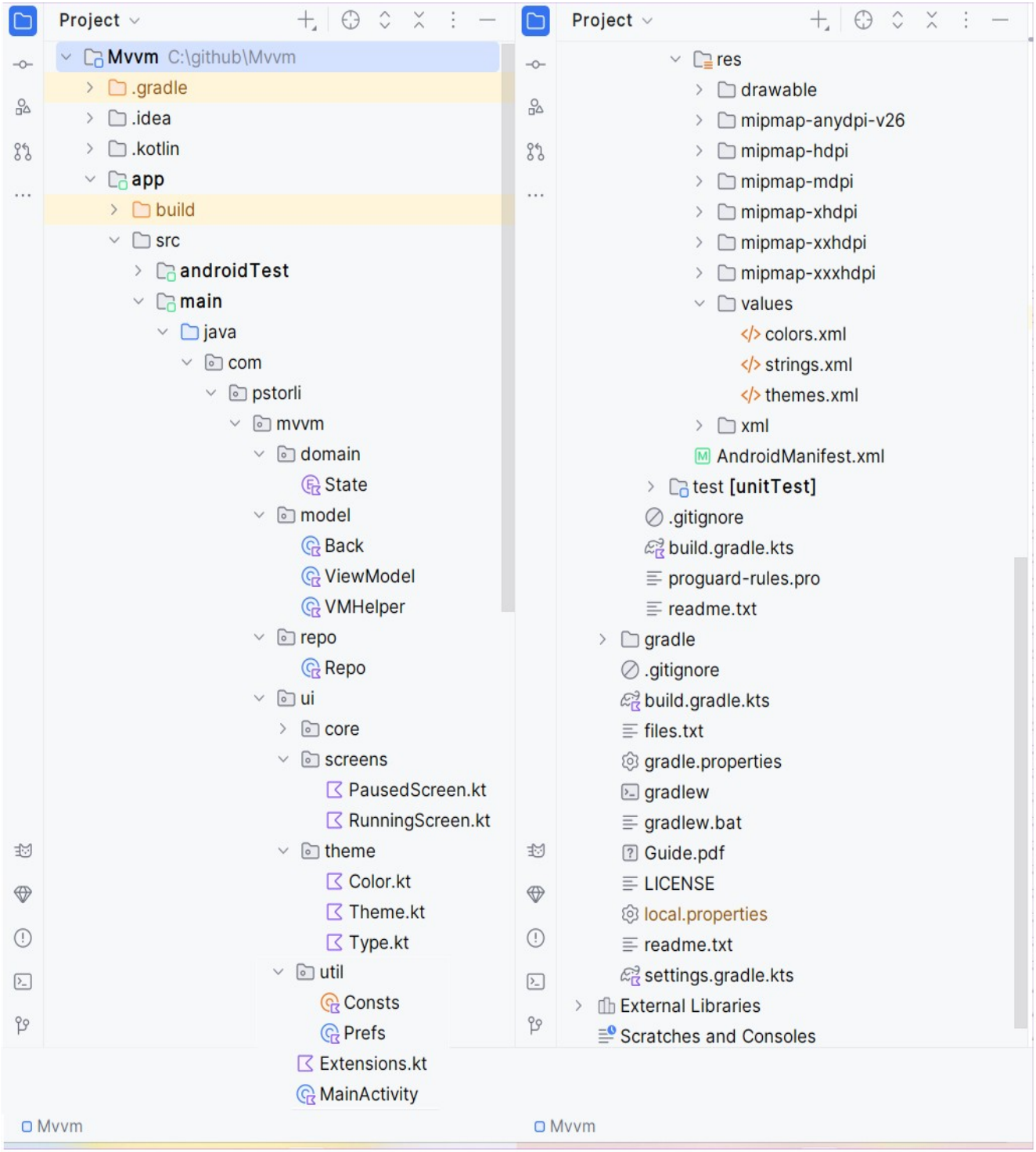
- Left click mouse far left in src file, breakpoint will appear as a red dot.
- Start program with bug icon, top left, of app or Run → Debug menu
- F7 step into, F8 step over, F9 run to next breakpoint
- when stopped at a breakpoint, you can view current state of variables.



Button toggles the **paused** and **running** states.
Paused state says "Start"
Running state says "Stop"

f) **File → Close Project**. The next section covers project architecture, program flow, **renaming project** and adding **it** to your own **github** account.

Android Studio Project Diagram



****

Main Packages

****

- **Domain package:**

  proj data. Currently we just have an enum class,

  State.kt – ***PAUSED*** and ***RUNNING***

- **model package:**

  handles data src and ui / viewmodel communication.

  Back.kt class handles viewmodel /repo communication, via coroutines, used to perform long running operations, that would otherwise **lock up the ui thread**.

  ViewModel.kt class has **viewmodel** which is data source, holds **repo** and **prefs**

  VMHelper.kt class holds viewmodel logic.

- **repo package**

  Repo.kt class for long running functions, done **not on ui thread**!

- **res package**

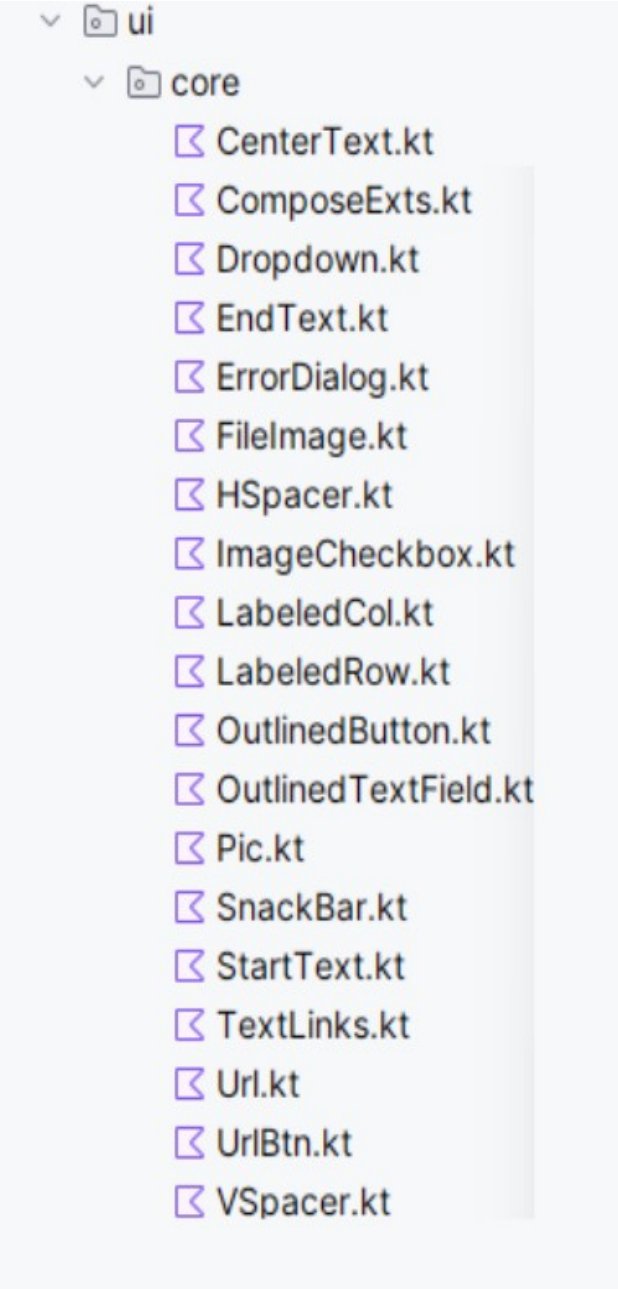
  holds resources, such as images and text strings

  drawable package Holds project image files, png and jpg

  values package Strings.xml file holds user displayable text, one file per language

· ui core package

holds custom, generic, composeable classes for project



Composeables are easy to use and make. Look at these examples in this directory for project independent composeables.

Pay careful attention to how the composeable is created, parameters passed, and how it indicates action, such as what a button press or click does.

The composeable, **OutlinedButton.kt**, is used as the sample button in the

composeable **RunningScreen.kt.**

Copy a composeable in core, play around with it, make new ones, they are re-usable gui, kind of like a reusable class in kotlin.

- **ui package**

  core package Custom, Generic, composeables

  screens package holds project's compose screens

    PausedScreen.kt A composeable sample start screen to show when in **Paused** state.

    RunningScreen.kt – A composeable to show when in **Running** state.

  theme package

    Theme.kt A composeable that handles light and **dark** color schemes.

- **util package holds utility classes**

  Prefs.kt class used for persisting data, when a database is overkill.

  Consts.kt class used for project wide constants.

****

Main Files

****

.gitignore

```
/**
 * This file has Files to not save into version control
 */
```

****

build.gradle.kts

```
// The namespace property defines the package name for your app's
// internal generated code like the R class
// (for resources) and BuildConfig class.
namespace = "com.pstorli.mvvm"

// unique identifier for your application on an Android device
applicationId = "com.pstorli.mvvm"

// What min and max android version do you support?
minSdk = 24
targetSdk = 36

// The version this app is on.
versionCode = 1
versionName = "1.0"

// This section is used to import external projects we need.
dependencies {}
```

****

Back.kt

```
/**
 * Back for run in background, not on ui thread.
 *
 * This class is used to spin off co-routines which call
 * long running functions in the Repo.kt class
 * NOTE: The repo class is private, we don't want it being
 * used by anyone but this class, which knows how to launch
 * co-routines.
 */
```

****

Color.kt

```
/**
 * This class holds the colors used in the app.
 */
```

****

colors.xml

```
/**
 * Hex color codes.
 */
```

****

Consts.kt

```
/**
 * A place to put project constants.
 */
```

****

Extensions.kt

/**
 * *A place to put class extensions.*
 */
A class extension allows you to add functionality to a class that it does not have. For example to add easy exception logging,

fun, then, the class, a period, then the function namespace

Like this:

```
fun Exception.logError ()
{
   Consts.logError (this.toString())
}

// This is how we use it.
   catch (ex: Exception) {
      ex.logError()
   }
```

****

MainActivity.kt

/**
This class is the applications main activity.

This is the only place we instantiate the ViewModel.kt class.
It is passed to composeables on their constructor.

The function, **onCreate**, in **MainActivity.kt**

```
override fun onCreate(savedInstanceState: Bundle?) {}
```

is called when program starts up and is on the ui thread, so no long running operations here, just call setContent to set the composeable to view,
In this case, **PausedScreen.kt**
*/

****

PausedScreen.kt

```
/**
 This is an example of a typical composeable.
Note that the viewModel is passed in on the constructor.

// This line both sets the background color, and listens for
// any future changes in the background color in the viewModel

.background (viewModel.backColor),

OutlinedButton (
   name        = viewModel.btnText,
   backColor   = Color (viewModel.btnBackColor.value),
   borderColor = Color.Red,

   // Toggle running state when clicked
   onClick     = {
       // Tell the view model to toggle the running state.
      viewModel.vh.toggleRunning()
   })
```

The **toggleRunning**() call, tells the viewModel that we want to toggle the running state. The ui should only handle two things, displaying the data and notifying the viewModel of ui actions.

We want to keep logic out of the ui, as much as possible.

```
 */
```

****

Prefs.kt

/**
 * This class can be used to store and retrieve data that persists, even when app and phone are turned off. The only way to “clean” or “clear” the preference data is by un-installing app.
 */

****

readme.txt

*/** A place to store info about developer, project and version changes.*
0000 Changes for Version X:
Changed …
*/

****

Repo.kt

/** This class handles talking to remote data sources to get data in the background, such as images from urls, websites, vpn, or even local database , which is considered slow.

This class is currently designed to be called from class Back.kt
so that it is never called directly from the ui thread.
If request is needed, add methods to Back.kt to get to the Repo.kt method you want to talk to.

The function oneTimeDataFetch, *normally does not need to be here, it is not long running.*
But it have it here as an example of how a longer running routine should be handled to fetch some data.

```
/**
   * Fetch a color.
   */
fun oneTimeDataFetch(): Color
```

The function executeBackgroundTask, is called repeatedly, in the background, off the ui thread.

```
/**
   * This function is called repeatedly, with delay, Consts.kt.
   * the background task,
   * which keeps running, periodically, between
   * Consts.MIN_DELAY and Consts.MAX_DELAY until
   * viewModel.state is PAUSED
   */
    fun executeBackgroundTask() {
```

****

RunningScreen.kt

/**
 This is an example of a typical composeable.
Note that the viewModel is passed in on the constructor.

This screen is shown when app in running state.

****

strings.xml

/**
 Where to put strings being displayed to the user.
  Useful for internationalization.
  Strings used for logging need not go here.
 */
****

Theme.kt

/**
 * Handles switching between dark and light modes for colors.
 */
****

themes.xml

/**
 * *Where to declare theme to use.*
 */

****

Type.kt

```
/**
 * Typography, font family, font size, font weight, line height and spacing
 */
```

****

ViewModel.kt

```
/**
   Holds project data, in a way that survives context switches,
    such as changing the phone's orientation
 */
```

****

VMHelper.kt

```
/**
 * This class is used for logic that is short lived, as we are on the ui thread
here. The reason is that I like to keep the viewModel as solely a data
repository, and as little logic, functions, as possible.
 */
```

Program starts in onCreate.

All android programs start in **MainActivity.kt**

```
override fun onCreate(savedInstanceState: Bundle?)
```

The main thing to do here is create the correct composeable,
inject it with the **viewModel** and set the content.

The **viewModel** holds the **data** for the entire app.

```
val viewModel: ViewModel by viewModels()
```

The content **should** be a **composeable**.
We pass the **viewModel** to the composeables constructor.

MvvmTheme.kt class handles colors the app uses in normal and dark mode.

```
setContent {
  MvvmTheme {
    Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
      // Show correct screen for game state.
      when (viewModel.state) {
        PAUSED  ->
          PausedScreen (
            viewModel =  viewModel,
            modifier  = Modifier.padding(innerPadding)
          )
        RUNNING ->
          RunningScreen (
            viewModel = viewModel,
            modifier  = Modifier.padding(innerPadding)
          )
```

Pressing the start or stop button.

The logic to handle the button press is in class **PausedScreen.kt** and **RunningScreen.kt**

```
Button (
    // Toggle running state when clicked
    onClick = {
        viewModel.vh.toggleRunning()
    },
```

When the user clicks the Start or Stop button, the routine **toggleRunning** is called in class **VMHelper.kt** **to change the viewModel.state**

We first update the state in class **ViewModel.kt**

Note: The **State.kt** class in enclosed by class **mutableStateOf**

```
var state by mutableStateOf<State>(State.PAUSED)
```

which, when modified, notifies the routine **onCreate**, in **MainActivity.kt**

```
when (viewModel.state)
```

With the following code in class **VMHelper.kt** :

```
// toggle running state
if (RUNNING == viewModel.state) {
    viewModel.state = PAUSED
}
else {
    viewModel.state = RUNNING
}
```

ToggleRunning

The next thing we do in the routine **toggleRunning** in class **VMHelper.kt** is call either **backgroundTaskCoroutine** or **dataFetchCoroutine** in class **Back.kt**

```
if (RUNNING == viewModel.state) {
    back.backgroundTaskCoroutine()
}
else {
    back.dataFetchCoroutine ()
}
```

Background Task Coroutine

The routine **backgroundTaskCoroutine** in class **Back.kt** causes
a background task, a co-routine, to start in view model scope:

```
bj = viewModel.viewModelScope.launch
```

The thread, co-routine, will keep running as long as the state is Running.

```
while (RUNNING == viewModel.state)
```

It repeatedly calls:

```
repo.executeBackgroundTask()
```

With a delay between calls.

```
// Wait 1 - 5 seconds
val time = (Consts.rndNum(MIN_DELAY, MAX_DELAY)).toLong()
"Back.backgroundTaskCoroutine Delaying: $time".logVerbose()

delay (time)
```

Note: The ***logVerbose*** call only logs if **Consts.kt VERBOSE is true.**
Use **logInfo** to always log events.

Execute Background Task

The routine **executeBackgroundTask** in class **Repo.kt** can do anything you want. It is called from the routine **backgroundTaskCoroutine** in class **Back.kt**

```
fun executeBackgroundTask()
```

It currently creates a new color,

```
val color = randomColor ()
```

and then sets the background color to the new color.

```
viewModel.backgroundColor = color
```

Data Fetch Coroutine

The routine **dataFetchCoroutine** in class **Back.kt** causes
a background task, a co-routine, to start in view model scope:

```
  viewModel.viewModelScope.launch
```

It calls the routine **oneTimeDataFetch** in ***Repo.kt***
one time only.

```
//  Create deferred var
  val fetchColorDeferred = viewModel.viewModelScope.async
(Dispatchers.Main)
{
    repo.oneTimeDataFetch ()
}

// Wait for it. new button color
val color = fetchColorDeferred.await()

// Go back contains ui thread.
// This causes us to rejoin the ui thread,
withContext (Dispatchers.Main) {

    // Update the button color in the view model.
    viewModel.backgroundColor = color
  }
```

One Time Data Fetch

The routine **oneTimeDataFetch** in class **Repo.kt** can do anything you want.
It is called from the routine **dataFetchCoroutine** in class **Back.kt**

```
fun oneTimeDataFetch() : Color
```

It currently creates a new color,

```
val color = randomColor ()
```

and then returns the new color to the
dataFetchCoroutine in class **Back.kt**

```
return color
```

Make Project your own.

Using this app, this sample project, as the foundation for any Android App that you want to create provides a solid base for any modern project.

a) Delete the current git repo

1) Open a file explorer, such as Windows Explorer

2) Go to the location where you downloaded
the "**MVVM**" application. *For me it was dir:* ***C:\github\MySample***

3) Delete the **.git** directory***:*** **C:\github\MySample\.git**
It is hidden, so you may need to show hidden files and dirs,
in your file explorer.

b) Rename project directories.

Note: ***mycompany, mysample*** *and* ***MySample****, are just suggested, names should be* ***case sensitive*** *and* ***have no spaces or other special characters****, so use whatever names you like. Use file explorer to:*

1) Rename company name directory:

from: C:\github**MySample**\app\src\main\java\com\pstorli

to: C:\github**MySample**\app\src\main\java\com**mycompany**

2) Rename project name directory:
from: C:\github**MySample**\app\src\main\java\com\pstorli**mvvm**
to: C:\github\MySample\app\src\main\java\com\pstorli**mysample**

3) The New project directory should look like:
C:\github**MySample**\app\src\main\java\com**mycompany\mysample**

c) Rename company and project name

1) Open a file editor, that can **search and replace multiple files**, such as **Notepad++**.

2) Open up file **C:\github\MySample\readme.txt**

Go to the location where you downloaded the "MySample" application. For me it was dir: C:\github\MySample

4) Replace the company name

Search → **Files in Files**

a) Match case true

b) Find What: pstorli

c) Replace With: **mycompany**

Replace in Files

5) Replace the project name

Search → **Files in Files**

a) Match case true

b) Find What: mvvm

c) Replace With: **mysample**

Replace in Files

6) Replace the project name, again

Search → **Files in Files**

a) Match case false

b) Find What: mvvm

c) Replace With: **MySample**

Replace in Files

d) Check yourself before you wreck yourself!

These files should have been modified by the previous steps.
If your editor cannot replace in multiple files at once,
you can change each file individuality.

I highly suggest you download **WinMerge** to help compare projects, or versions of same project.

https://winmerge.org/downloads

*Use this list to spot check your progress. Items in **Green** below should be your new company or project name.*

C:\github\MySample\app\src\main\res\values\strings.xml

```
<string name="app_name">MVVM</string>
```

C:\github\MySample\app\build.gradle.kts

```
android {
    namespace = "com.pstorli.mvvm"
        defaultConfig {
            applicationId = "com.pstorli.mvvm"
```

C:\github\MySample\app\src\main\AndroidManifest.xml

```
    <application
        android:theme="@style/Theme.Mvvm">
            <activity
                android:theme="@style/Theme.Mvvm ">
```

??? These files may be modified. ???

C:\github\MySample\.idea**workspace.xml**
C:\github\MySample\.idea**.name**

e) Open up your new project in Android Studio

1) Projects → Open

2) **C:\github\MySample**

3) Settings → Version Control → Directory Mappings

3) Invalidate caches and restart.

4) Reopen Project, if it doesn't open automatically.

f) Update app icon.

1) Right click the **res** directory
2) Select New → Image Asset.
3) Ensure **Icon Type** is set to **Launcher Icons (Adaptive and Legacy)**.
4) Set Source Asset, Asset Type: **Image**

5) Set Source Asset, Path: ***Path to your 512x512 icon.png***

6) Click Next → Finish

g) Integrate version control with github

Share Project on github: **(Version Control System)**
Go to the menu bar and **select**

Git → GitHub → Share Project on GitHub